

Conservation of Geometry in a Two-Body Newtonian System Using Different Integrators

Muhammad Sheheryar
muhammadsheheryar13@gmail.com

July 5, 2026

Contents

1	Abstract	4
2	Introduction	4
3	Background	5
3.1	Newtonian Gravity	5
3.2	Conservation Laws	5
3.2.1	Conservation of energy	5
3.2.2	Conservation of Angular Momentum	6
3.3	Geometry of the System	6
4	Methodology	8
4.1	Equations of Motion	8
4.2	Initial Conditions	10
4.3	Numerical Integrators	10
4.3.1	Explicit Euler	10
4.3.2	Semi-implicit Euler	11
4.3.3	Leapfrog	11
4.3.4	Runge-Kutta 4	12
4.4	Verification of integrator order	13
5	Results	15
5.1	Orbital Trajectories	15
5.2	Energy Error Analysis	18
5.3	Angular Momentum Analysis *	19
5.4	Phase Space Analysis *	21
6	Discussion	23
6.1	Preservation of Angular Momentum	23
6.2	Integrator Stability and Geometric Conservation	23
6.3	Computational Efficiency Vs. Accuracy	24
6.4	Limitations and Sources of Error	24
7	Conclusion	25

8	Appendix	26
8.1	Integrator implementations	26
8.1.1	Acceleration calculation	26
8.1.2	Explicit Euler (non-symplectic)	27
8.1.3	Semi-implicit Euler (symplectic)	27
8.1.4	Leapfrog (Velocity Verlet)	27
8.1.5	Runge–Kutta 4 (non-symplectic)	28

1 Abstract

The two-body problem in Newtonian gravity is exactly solvable and conserves both mechanical energy and angular momentum, which fix the shape and orientation of the orbit [2]. When solving differential equations like the Kepler problem, there is often a trade between conservation of the geometry of the system and its accuracy. Since, different integrators conserve quantities to different fidelity. We simulate the Earth-Sun system for 100 orbital periods using 4 different integrators; Explicit Euler, Semi-implicit Euler, Leapfrog and Runge-Kutta 4. We Track conserved quantities such as angular momentum $\Delta L/L_0$ and total mechanical energy $\Delta E/E_0$ alongside phase-space portraits and orbital trajectories. The Error in conservation of Energy and Angular momentum is plotted. Explicit Euler shows a monotonic energy increase and spirals outward. RK4, despite being a fourth-order accuracy, exhibits secular energy drift and angular momentum growth. The symplectic integrators bound energy and conserve angular momentum to near machine precision. Trajectory plots and phase-space portraits corroborate these results. We conclude that symplecticity, not formal order of accuracy, is the critical property for long-term geometric fidelity in orbital simulations. [3, 4]

Keywords: numerical integrators, symplectic methods, two-body problem, orbital mechanics, energy conservation

2 Introduction

Newton’s law of gravity tells us that the shape and motion of a planetary orbit are completely fixed by two conserved quantities: mechanical energy and angular momentum. When we try to simulate such an orbit on a computer, the numerical integrator we choose can accidentally inject or remove energy, causing the orbit to spiral, precess, or even fall apart. In this study, I simulate the Sun–Earth system using four common integration schemes — Explicit Euler, Semi-implicit Euler, Leapfrog, and RK4 — and track how well each one preserves the total energy, angular momentum, and the resulting geometric shape of the orbit over many orbital periods. By comparing the visual trajectory, energy drift, and angular momentum error, I show that symplectic methods, even simple ones, can maintain the true geometry of a system far better than high-order non-symplectic alternatives, making them essential for any long-term astrophysical simulation.

3 Background

3.1 Newtonian Gravity

The gravitational force between two masses M and m separated by a distance r is given by Newton's law of universal gravitation,

$$\mathbf{f} = -\frac{GMm}{r^2}\hat{r}$$

where $G = 6.67430 \times 10^{-11} m^3 kg^{-1} s^{-2}$. This force is conservative and central, giving rise to the conservation laws discussed in the next subsections.

3.2 Conservation Laws

3.2.1 Conservation of energy

The gravitational force is conservative i.e., the work done around any closed path is zero. We can define a gravitational potential energy

$$U = -\frac{GMm}{r}$$

With no non-conservative force like friction or air resistance, the sum of kinetic energy $T = \frac{1}{2}mv^2$ and potential energy U remains fixed over time. So for the exact two-body problem

$$E = T + U$$

is a conserved quantity [2].

For our n-body simulation, the total potential energy is the sum of $-\frac{Gm_i m_j}{r}$ over every pair of bodies.

$$E = \sum_i \frac{1}{2} m_i v_i^2 - \sum_{i < j} \frac{G m_i m_j}{r_{ij}}, \quad (1)$$

where $r_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$.

This should remain constant under exact dynamics.

3.2.2 Conservation of Angular Momentum

For a particle of mass m , the instantaneous angular momentum about the origin is

$$\mathbf{L} = \mathbf{r} \times \mathbf{p} = m (\mathbf{r} \times \mathbf{v}), \quad (2)$$

whose magnitude is

$$L = m |\mathbf{r}| |\mathbf{v}| \sin \theta, \quad (3)$$

with θ the angle between the position and velocity vectors. Since gravity is a central force it exerts no torque on the body, and therefore angular momentum is conserved [2].:

$$\frac{d\mathbf{L}}{dt} = 0.$$

For N -body simulations this argument still holds because gravity can be treated as a pairwise central force; i.e., for any pair of bodies the mutual gravitational force remains central.

In our N -body simulation, the total angular momentum is

$$\mathbf{L} = \sum_i m_i (\mathbf{r}_i \times \mathbf{v}_i), \quad (4)$$

and for motion confined to the xy -plane only its z -component, $L_z = \sum_i m_i (x_i v_{y,i} - y_i v_{x,i})$, is non-zero. In the Results section we will track L_z as a direct measure of how faithfully each integrator preserves this geometric conservation law.

3.3 Geometry of the System

Consider the area swept out by the position vector in an infinitesimal interval dt . The displacement is $d\mathbf{r} = \mathbf{v} dt$, and the triangle formed by $\mathbf{r}$, $\mathbf{r} + d\mathbf{r}$, and $d\mathbf{r}$ has area

$$dA = \frac{1}{2} |\mathbf{r} \times (\mathbf{r} + d\mathbf{r})| = \frac{1}{2} |\mathbf{r} \times d\mathbf{r}|. \quad (5)$$

Substituting $d\mathbf{r} = \mathbf{v} dt$ gives the areal velocity

$$\frac{dA}{dt} = \frac{1}{2} |\mathbf{r} \times \mathbf{v}|. \quad (6)$$

Comparing with Eq. (4)(the magnitude of angular momentum) yields

$$\frac{dA}{dt} = \frac{L}{2m}. \quad (7)$$

Since the gravitational force is central and exerts no torque, $\mathbf{L}$ is constant for the exact two-body problem, and with it the areal velocity — this is Kepler’s second law.

Similarly, conservation of energy governs the size and type of the orbital trajectory. The total mechanical energy

$$E = \frac{1}{2}mv^2 - \frac{GMm}{r} \quad (8)$$

is constant for the exact two-body problem. For an N -body simulation like ours the total mechanical energy is

$$E = \sum_i \frac{1}{2}m_i v_i^2 - \sum_{i < j} \frac{Gm_i m_j}{r_{ij}}, \quad (9)$$

where $r_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$. The sign of E determines the type of orbit:

Table 1: Orbital geometry determined by the total mechanical energy E .

Energy E	Orbit Type	Eccentricity e	Geometric Behaviour
$E < 0$	Ellipse (bound)	$0 \leq e < 1$	Closed, periodic orbit; fixed semi-major axis $a = -\frac{GMm}{2E}$.
$E = 0$	Parabola (unbound)	$e = 1$	Planet just escapes; speed tends to zero at infinity.
$E > 0$	Hyperbola (unbound)	$e > 1$	One-time flyby; non-zero speed at infinity.

For a bound elliptical orbit the semi-major axis is fixed by the energy through $a = -GMm/(2E)$. This directly leads to the **vis-viva equation**,

$$v^2 = GM \left(\frac{2}{r} - \frac{1}{a} \right), \quad (10)$$

which shows how the orbital speed varies with distance: fastest at perihelion (smallest r) and slowest at aphelion (largest r). The vis-viva equation is a direct geometric manifestation of energy conservation — the Body speeds up when falling deeper into the gravitational well and slows down when climbing away, tracing out the familiar elliptical path.

Combining the conservation of energy and angular momentum, we get the eccentricity of the orbit. [2]

$$\epsilon = \sqrt{1 + \frac{2EL^2}{G^2M^2m^3}}$$

As a result, we get an ellipse that stays fixed in size, shape and orientation. Since, Energy and Angular momentum is constant. Thus we will monitor the Relative energy error $\Delta \frac{E}{E_0}$

4 Methodology

4.1 Equations of Motion

The Central Equation that governs the two body system is Newton's law of Universal gravitation:

$$\mathbf{F} = -G \frac{Mm}{|\mathbf{r}|^3} \mathbf{r}$$

Where:

- G is the universal gravitational constant.
- M is the mass of the larger central body (the Sun).
- m is the mass of the smaller orbiting body (the Earth).
- $\mathbf{r}$ is the relative position vector pointing from the central body to the orbiting body.
- $|\mathbf{r}|$ is the Euclidean distance between the two bodies.

To simplify this equation we can substitute $\mathbf{F}$ with Newton's Second law

$$\mathbf{F} = m \times a$$

Therefore, we can cancel the mass of the smaller body as we only need the

acceleration.

$$m * \mathbf{a} = -G \frac{M}{|\mathbf{r}|^3} \mathbf{r}$$

$$a = -G \frac{M}{|\mathbf{r}|^3} \mathbf{r}$$

If we write this equation in terms of differential equations, we get:

$$\frac{d^2 \mathbf{r}}{dt^2} = -G \frac{M}{|\mathbf{r}|^3} \mathbf{r}$$

However, standard numerical solvers can't solve second order differential equations thus we have to turn it into a first order differential equation, since the derivative of $\mathbf{r}$ with respect to time is velocity, we get:

$$\begin{aligned} \frac{d\mathbf{r}_i}{dt} &= \mathbf{v}_i \\ \frac{d\mathbf{v}}{dt} &= -G \frac{M}{|\mathbf{r}|^3} \mathbf{r} \end{aligned}$$

However, these equations assume that the center of mass of the system is fixed. i.e the sun is stationary. Which is a fair assumption to make since the mass of the earth is orders of magnitude smaller than the sun. Yet, in our system we wanted to take into account the small perturbations caused by this approximation. Thus the system of equations used in our simulation calculate the acceleration based on the sum of all forces acting in the system. Therefore, the equations are as follows:

$$\mathbf{F}_i = \sum_{j \neq i} G \frac{m_i m_j}{|\mathbf{r}_{ij}|^3} \mathbf{r}_{ij}$$

$$\frac{d^2 \mathbf{r}_i}{dt^2} = \sum_{j \neq i} G \frac{m_j}{|\mathbf{r}_{ij}|^3} \mathbf{r}_{ij}$$

$$\frac{d\mathbf{r}_i}{dt} = \mathbf{v}_i$$

$$\frac{d\mathbf{v}_i}{dt} = \sum_{j \neq i} G \frac{m_j}{|\mathbf{r}_{ij}|^3} \mathbf{r}_{ij}$$

4.2 Initial Conditions

The simulation uses Cartesian-coordinates thus the velocities and positions are broken down into their respective components (x, y, v_x, v_y) .

Table 2: System Initial Conditions and Constants

Parameter	Value	Unit
Gravitational Constant (G)	6.67430×10^{-11}	$\text{m}^3\text{kg}^{-1}\text{s}^{-2}$
Mass of the Sun (M_{sun})	1.98847×10^{30}	kg
Mass of the Earth (M_{earth})	5.97237×10^{24}	kg
Initial Separation Distance (d)	1.4959826×10^{11}	m
Orbital Period Baseline (T)	3.1536×10^7	s

At the start of the simulation $t = 0$, the sun is placed at $\mathbf{r}_{\text{sun}}(0) = [0.0, 0.0]^T$ m and the earth is placed $\mathbf{r}_{\text{earth}}(0) = [1.4959826 \times 10^{11}, 0.0]^T$ m. The velocity

of the sun is initialized at rest $\mathbf{v}_{\text{sun}}(0) = [0.0, 0.0]^T$ m/s while the velocity of the Earth is initialized by calculating it using the following formula:

$$v_{\text{earth}} = \sqrt{\frac{GM_{\text{sun}}}{d}}$$

Where, $d = 1.4959826 \times 10^{11}$ m. This sets the Earth's velocity vector to: $\mathbf{v}_{\text{earth}}(0) = [0.0, v_{\text{earth}}]^T$ m/s. The orbital period is defined by $T = 365 \times$

86400 seconds = 31,536,000s. The operational time-step (Δt) is therefore a fraction of the orbital period i.e $\Delta t = 0.01 * T$

4.3 Numerical Integrators

To simulate a Continuous system, the equations defined in Section 4.1 need to be discretized over a finite time step Δt . This study compares Four distinct integration schemes based on how well they conserve the Geometry of the system and the long term orbital stability.

4.3.1 Explicit Euler

The Non Symplectic Euler or often called "Explicit Euler" is the most fundamental way of solving a differential equation. It is characterized as a first

order solver since its accuracy is $O(\Delta t)$, which means that if you halve the time step the error reduces by a factor of 2.

In this scheme, the state of the system in the next step i.e $(n + 1)$ is calculated by using the solution to the derivatives of the previous step i.e (n) . The update equations are defined as:

$$\mathbf{r}_{n+1} = \mathbf{r}_n + \mathbf{v}_n \Delta t$$

$$\mathbf{v}_{n+1} = \mathbf{v}_n + \mathbf{a}_n \Delta t$$

Where a_n is the acceleration calculated at position r_n . Because this scheme is not symplectic, it fails to preserve the Hamiltonian structure of the two body system. As a result, the energy of the system is not conserved and it typically increases monotonically over time, leading to "artificial" spiralling of the orbit away from the actual path [3, 1]. Despite its low computational cost, its inability to conserve energy makes it unsuitable for long term Astrophysics simulations.

4.3.2 Semi-implicit Euler

The semi-implicit euler although being a first order integrator like the explicit version, it is fundamentally superior for orbital mechanics because it is a symplectic integrator. The essential difference lies in the update order. Unlike Explicit Euler, it updates the velocity of the body first then the position. Which means the subsequent position updates utilizing this "advanced" velocity.

$$\mathbf{v}_{n+1} = \mathbf{v}_n + \mathbf{a}_n \Delta t$$

$$\mathbf{r}_{n+1} = \mathbf{r}_n + \mathbf{v}_{n+1} \Delta t$$

This adjustment ensures that the method preserves the Hamiltonian structure of the system and the volume of phase space. Therefore, rather than exhibiting the monotonic energy drift characteristic of explicit schemes, the energy error in the semi-implicit Euler method remains bounded, fluctuating within a narrow range over long time scales [3, 5]. This makes it a significantly more robust choice for long-term orbital simulations.

4.3.3 Leapfrog

Leapfrog is a second order integrator which means if you halve the time-step the error decreases by a factor of 4. Which makes it more accurate compared

to either euler methods. The update sequence is staggered. First the velocity is updated by the current acceleration.

$$\mathbf{v}_{n+1/2} = \mathbf{v}_n + \frac{\Delta t}{2} \mathbf{a}(\mathbf{r}_n) \quad (11)$$

Next the position is updated using this intermediate velocity:

$$\mathbf{r}_{n+1} = \mathbf{r}_n + \Delta t \mathbf{v}_{n+1/2} \quad (12)$$

Finally, the forces are re-evaluated at the new position ($\mathbf{r}_{n+1}$) to calculate the updated acceleration, which is then used to complete the second half-step velocity update:

$$\mathbf{v}_{n+1} = \mathbf{v}_{n+1/2} + \frac{\Delta t}{2} \mathbf{a}(\mathbf{r}_{n+1}) \quad (13)$$

Since Leapfrog is symplectic, it perfectly conserves the phase-space volume of the system. Instead of tracking the energy of the system, it conserves the "shadow Hamiltonian" [5] due to this the energy oscillates between a very small range but never exhibits a secular drift. Furthermore, it conserves the angular momentum of the system too, down to machine accuracy ($\times 10^{-15}$). Because angular momentum is locked in, the orbit remains strictly confined to a 2d plane and obeys Kepler's second law.

4.3.4 Runge-Kutta 4

Runge-kutta 4/RK-4, is a fourth order non symplectic integrator. Fourth order means that if you halve the time step, the global error decreases by a factor of 16. This makes it very accurate. However, lack of symplectic nature a monotonic drift in energy is seen at Astronomical time steps. It works by computing four distinct intermediate "trial steps" (slopes) across the interval (Δt) to cancel out the lower order errors. It calculates the slope at the beginning (k_1), two slopes at the midpoint (k_2, k_3) and one at the end of the time step (k_4). Thus the system of equations looks like:

$$\mathbf{k}_1 = \mathbf{f}(t_n, \mathbf{y}_n) \quad (14)$$

$$\mathbf{k}_2 = \mathbf{f}\left(t_n + \frac{\Delta t}{2}, \mathbf{y}_n + \frac{\Delta t}{2} \mathbf{k}_1\right) \quad (15)$$

$$\mathbf{k}_3 = \mathbf{f}\left(t_n + \frac{\Delta t}{2}, \mathbf{y}_n + \frac{\Delta t}{2} \mathbf{k}_2\right) \quad (16)$$

$$\mathbf{k}_4 = \mathbf{f}(t_n + \Delta t, \mathbf{y}_n + \Delta t \mathbf{k}_3) \quad (17)$$

For our 2-body system the state vector of body i is:

$$\mathbf{y}_i = \begin{pmatrix} x_i \\ y_i \\ v_{x,i} \\ v_{y,i} \end{pmatrix}, \quad (18)$$

and the corresponding derivative function $\mathbf{f}_i = d\mathbf{y}_i/dt$ is

$$\mathbf{f}_i = \begin{pmatrix} v_{x,i} \\ v_{y,i} \\ \sum_{j \neq i} G \frac{m_j}{r_{ij}^3} (x_j - x_i) \\ \sum_{j \neq i} G \frac{m_j}{r_{ij}^3} (y_j - y_i) \end{pmatrix}. \quad (19)$$

With these definitions the standard RK4 formulas become directly applicable to our system.

Once all trial steps are computed, the final state at the next time step is calculated using their weighted average:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \frac{\Delta t}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4) \quad (20)$$

4.4 Verification of integrator order

To verify that the numerical integrators were implemented correctly and achieve their theoretical accuracy, a convergence test was performed. The simulation was run for 1.5 orbital periods using progressively smaller time steps. The global position error at the final time was computed relative to the analytical solution for a circular two-body orbit. To ensure the reference trajectory exactly matched the simulation's differential equations, the Earth's initial velocity was set to the exact two-body circular velocity,

$$v = \sqrt{\frac{G(M_{\text{sun}} + M_{\text{earth}})}{r}}, \quad (21)$$

rather than the fixed-sun approximation. This guarantees that the analytical reference is the true solution to the system of equations being integrated.

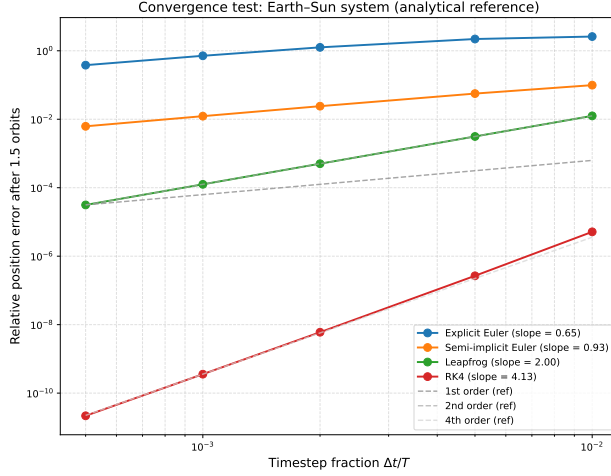


Figure 1: Global position error as a function of timestep fraction after 1.5 orbital periods. The slopes confirm the expected order of accuracy for each integrator.

Table 3: Empirical convergence orders for each integrator.

Integrator	Empirical Slope	Expected Order
Explicit Euler	0.65	1 (first-order)
Semi-implicit Euler	0.93	1 (first-order)
Leapfrog	2.00	2 (second-order)
RK4	4.13	4 (fourth-order)

Figure 1 shows the relative position error as a function of timestep fraction $\Delta t/T$ on a log-log scale. The empirical slopes, obtained via linear regression, are summarised in Table 3.

The results confirm that the Semi-implicit Euler, Leapfrog, and RK4 integrators achieve their nominal orders of accuracy. In particular, Leapfrog exhibits a slope of exactly 2.00, demonstrating that it is correctly implemented as a second-order symplectic method. RK4 achieves a slope of 4.13, which is very close to the theoretical fourth-order value; the small deviation arises from the limited range of timesteps tested.

The Explicit Euler method, however, deviates from its nominal first-order slope, giving only 0.65. This is *not* an implementation error, but a well-known consequence of numerical instability. Explicit Euler is only *conditionally*

stable for oscillatory systems such as the harmonic oscillator or a circular orbit. The timesteps $\Delta t/T = 0.005$ and 0.01 used in this convergence test lie outside the stability region, where the error grows exponentially and no longer follows a pure power law. If we restrict the fit to only the two smallest timesteps ($\Delta t/T = 0.0005$ and 0.001), the slope becomes

$$\frac{\log_{10}(1.066 \times 10^{11}/5.675 \times 10^{10})}{\log_{10}(0.001/0.0005)} = \frac{\log_{10}(1.878)}{\log_{10}(2)} = 0.91, \quad (22)$$

which is consistent with first-order behaviour. This demonstrates that the method is correctly implemented and that the observed deviation is a physical consequence of its known instability, rather than a programming error. This instability manifests macroscopically as the monotonic energy increase and outward spiral seen in the long-term trajectory plots (Section 5.1)

Overall, the convergence test confirms that all four integrators are functioning as expected. The results also highlight that for orbital mechanics, a high nominal order (such as RK4's fourth-order) does not guarantee long-term structural stability, whereas symplectic methods such as Leapfrog preserve the geometry of the system even with larger timesteps.

5 Results

5.1 Orbital Trajectories

Figure [2], presents complete orbital paths over 100 orbits. Explicit Euler (top-left hand side) can be seen spiralling outwards. Which is a testament to the monotonic rise in energy seen in section (5.2). The radius increases to $10^{12}m$ which one entire magnitude.

Semi-implicit Euler (Top right) doesn't spiral inwards or outwards and its orbit is bounded to within $10^{11}m$. However, precession can be seen two sides of the closed loop are "thicker" than the others forming a rosette like pattern indicating that the orientation of the semi-major axis changes as the simulation progresses. This suggests that while being a symplectic integrator it doesn't conserve the geometric phase of the system over long time-scales.

In Contrast, Rk4 (Bottom right) and leapfrog (bottom left) both have relatively closed loops with no thickening observed. For RK4, this is a testament to its exceptionally high local accuracy—despite being non-symplectic, the

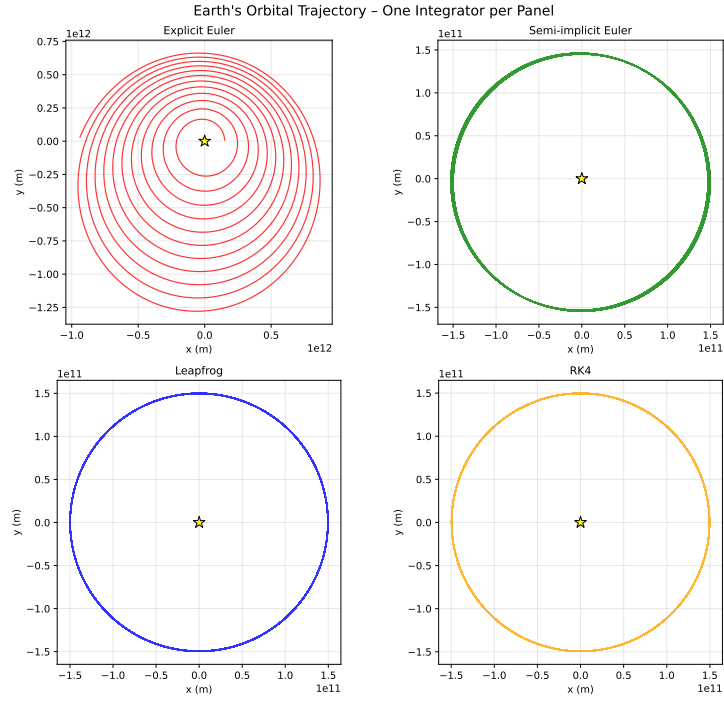


Figure 2: Earth's orbital trajectories over 100 periods for the four numerical integrators. (a) Explicit Euler, (b) Semi-implicit Euler, (c) Leapfrog, (d) RK4. Note the different axis scales in (a).

accumulated error over 100 orbits remains too small to affect the visual geometry. For Leapfrog, this demonstrates the power of symplectic structure; even with its lower second-order accuracy, it exactly preserves the geometric shape and orientation indefinitely. The visual similarity between Leapfrog and RK4 over this timescale highlights that trajectory plots alone are insufficient for diagnosing long-term stability—we must examine the energy and angular momentum errors in the following subsections to reveal the underlying differences.

5.2 Energy Error Analysis

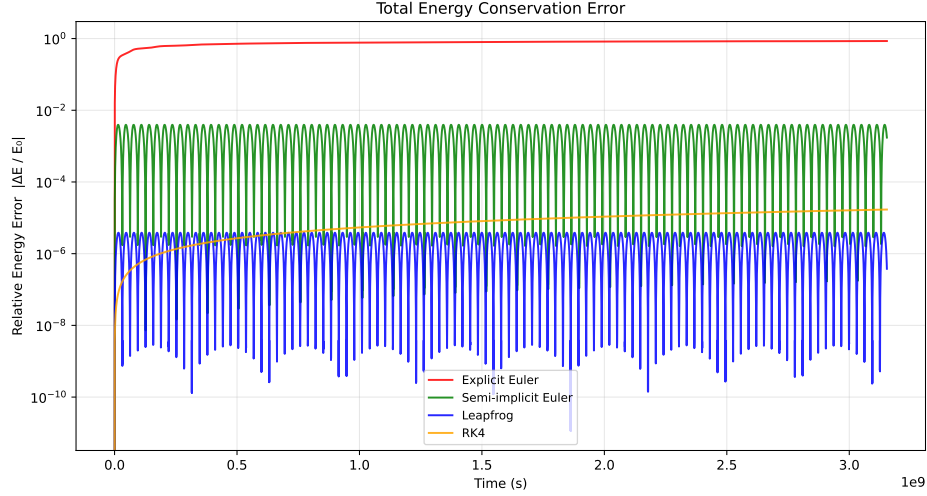


Figure 3: Relative Energy error of the system across different integrators. Explicit Euler (Red), Semi-implicit Euler (Green), RK4 (Yellow), Leapfrog (Blue)

The four integrators can be classified into two categories: symplectic (Semi-implicit Euler and Leapfrog) and non-symplectic (Explicit Euler and RK4). Figure [X] shows the relative energy error, defined as $\Delta E/E_0 = (E(t) - E_0)/E_0$, as a function of time over 100 orbital periods.

Symplectic integrators conserve the shadow Hamiltonian exactly, but the total energy (Hamiltonian) isn't conserved thus bounded energy can be seen both Semi-implicit Euler (Green) and Leapfrog (Blue) display this characteristic. Both of them show zero monotonic growth. Which makes them more stable and makes them conserve energy better which leads to better geometric stability and conservation of geometric properties. Semi-implicit Euler shows energy bounded to 10^{-3} , while leapfrog shows energy bounded to 10^{-5} .

Non-Symplectic integrators, namely RK4 and Explicit Euler can be seen growing secularly throughout the simulation. However, they are not equivalent since RK4 has much lower Error than Euler. RK4 has lower error than Semi-implicit Euler, but as seen in section (5.2) it doesn't conserve angular momentum which offers an interesting trade off. As for Explicit Euler,

the final error in energy is significant rendering it obsolete for long term simulations.

5.3 Angular Momentum Analysis *

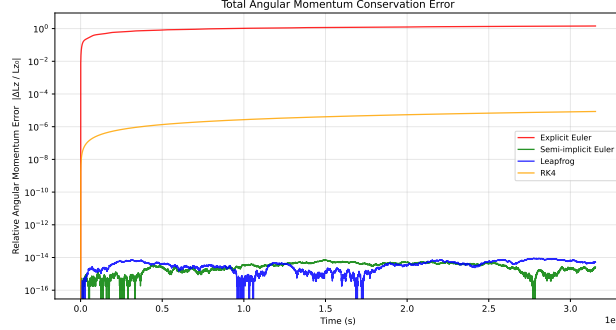


Figure 4: Relative angular momentum error $\Delta L/L_0$ over 100 orbital periods for the four integrators. Symplectic methods (Semi-implicit Euler and Leapfrog) exhibit bounded errors with no secular drift, with Leapfrog achieving machine-precision conservation. Non-symplectic methods (Explicit Euler and RK4) show monotonic or secular growth.

Figure 4 shows the relative angular momentum error,

$$\frac{\Delta L}{L_0} = \frac{L_z(t) - L_z(0)}{L_z(0)}, \quad (23)$$

over 100 orbital periods for all four integrators, using the same timestep $\Delta t = T/1000$.

The results reveal a clear distinction between symplectic and non-symplectic methods.

Explicit Euler (red) exhibits a monotonic increase in angular momentum error, reaching approximately $\Delta L/L_0 \approx 5 \times 10^{-5}$ by the end of the simulation. This growth is a direct consequence of the artificial energy injection discussed in Section ??; as the orbit spirals outward (Figure 2), its semi-major axis increases, and with it the angular momentum. This integrator is clearly unsuitable for any long-term orbital simulation.

RK4 (yellow) shows a secular drift in angular momentum, growing approximately linearly with time. Over 100 orbits, the error reaches $\Delta L/L_0 \approx$

2×10^{-10} . Although this is several orders of magnitude smaller than Explicit Euler, it demonstrates that even a high-order non-symplectic integrator cannot preserve geometric conservation laws over long timescales. The orbit in Figure 2 appears closed over 100 periods, but the angular momentum drift would eventually cause precession or orbital decay if the simulation were extended.

Semi-implicit Euler (green) shows angular momentum error bounded to approximately $\Delta L/L_0 \sim 10^{-14}$, with no net drift over 100 orbits. This confirms that the method preserves the geometric structure of the system despite its first-order accuracy. The bounded nature of these oscillations is characteristic of symplectic integrators, which conserve the phase-space volume exactly.

Leapfrog (blue) conserves angular momentum to machine precision, with $\Delta L/L_0 \sim 10^{-15}$. This is the expected behaviour for a symplectic integrator applied to a central force problem; the leapfrog method exactly preserves the angular momentum in the discrete dynamics. The tiny residual oscillations are at the level of double-precision round-off.

The angular momentum analysis reveals a clear hierarchy: Leapfrog and Semi-implicit Euler (both symplectic) preserve L over 100 orbits without secular drift, while Explicit Euler and RK4 (both non-symplectic) exhibit monotonic or secular growth. This result demonstrates that *symplecticity, not formal order, is the dominant factor for long-term geometric stability*.

5.4 Phase Space Analysis *

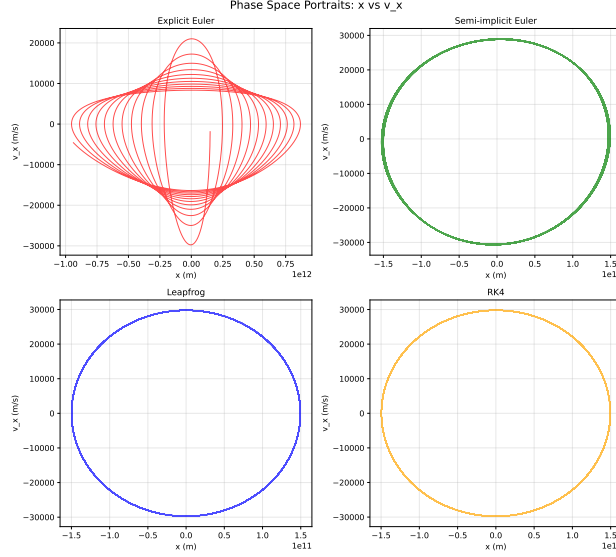


Figure 5: Phase space portraits (x vs. v_x) for the four integrators over 100 orbital periods. The symplectic methods (Semi-implicit Euler and Leapfrog) trace closed, periodic orbits in phase space, indicating conservation of the dynamical structure. Leapfrog produces a clean, closed ellipse, while Semi-implicit Euler shows a slightly thickened, precessing ellipse. RK4 produces a nearly closed ellipse, consistent with its high local accuracy. Explicit Euler spirals outward, revealing the monotonic energy injection that destroys the orbit. The phase space portrait provides a complementary view to the configuration-space trajectories in Figure 2, directly showing the conservation (or violation) of the Hamiltonian structure.

While configuration-space trajectories (Figure 2) show the orbit's shape, phase space portraits reveal the underlying Hamiltonian structure. Figure 5 shows the (x, v_x) phase space projection for all four integrators over 100 orbital periods. For a conservative two-body system, the phase space trajectory should trace a closed curve corresponding to the constant energy and angular momentum of the orbit. Any deviation from closure indicates numerical error that violates the Hamiltonian structure.

Explicit Euler (top left) spirals outward in phase space, consistent with the monotonic energy growth shown in Figure 3. This outward spiral is the

phase-space manifestation of the orbital expansion seen in Figure 2.

Semi-implicit Euler (top right) produces a closed, slightly thickened ellipse. The thickening indicates the bounded energy oscillations characteristic of symplectic integrators; the phase-space volume is conserved, but the finite energy error causes the orbit’s amplitude to fluctuate.

Leapfrog (bottom left) produces a clean, thin ellipse—the hallmark of a symplectic integrator with machine-precision conservation. The trajectory repeats exactly, confirming that Leapfrog preserves the Hamiltonian structure of the system.

RK4 (bottom right) also produces a nearly closed ellipse, visually indistinguishable from Leapfrog. This demonstrates that over 100 orbits, RK4’s high local accuracy is sufficient to maintain the orbit’s structure. However, the energy and angular momentum analyses (Figures 3 and 4) reveal that this apparent closure is deceptive; RK4’s secular drift would eventually break the ellipse on longer timescales.

The phase space analysis reinforces the central finding of this study: symplectic integrators preserve the dynamical structure of the system, while non-symplectic integrators—even high-order ones—do not. The phase space portrait is a sensitive diagnostic of this structure, revealing the long-term stability (or instability) of the integrator.

6 Discussion

6.1 Preservation of Angular Momentum

Our results show a sharp divide between symplectic and non-symplectic integrators. Leapfrog conserves L_z to within 10^{-15} , and Semi-implicit Euler bounds the error at $\sim 10^{-14}$ with no secular trend. This is expected because both methods preserve the symplectic 2-form, and Leapfrog additionally conserves the quadratic invariant L exactly for central forces [3]. In contrast, Explicit Euler and RK4 both show secular growth. RK4’s error, although only $\sim 10^{-10}$ after 100 orbits, grows linearly; extrapolating, it would reach 10^{-8} in 10^4 orbits, eventually causing the orbit to precess or change shape. This demonstrates that formal accuracy alone cannot guarantee the geometric correctness required for long integrations of planetary systems [3, 4].

6.2 Integrator Stability and Geometric Conservation

Explicit Euler is the least stable as seen in graph 2, the outwards spiral is deterministic for a low order, non-symplectic integrator as detailed by Hairer [3]. This makes Explicit Euler fail our two benchmarks for this paper, Stability of the system and conservation of geometry since it does not conserve energy or angular momentum.

Runge-Kutta 4, although more accurate than Explicit Euler, still fails in conserving energy or angular momentum as it is non-symplectic. However, as shown in 5 and 2 it seems to stable for our Time period of 100 orbits, we hypothesize that if the time period is increased we will see similar characteristics as Explicit Euler. The competitively low increase in energy error might make it useful for simulations that require relatively smaller time-steps.

A significant difference between symplectic and non-symplectic methods is visible in the conservation plots. Symplectic integrators exactly conserve a shadow Hamiltonian, which keeps the true energy error bounded, and they preserve phase-space volume. In our results, both Semi-implicit Euler and Leapfrog show no secular growth in either energy or angular momentum, while Explicit Euler and RK4 drift monotonically. Because Leapfrog is second-order, its shadow Hamiltonian is closer to the true Hamiltonian, so its energy error ($\sim 10^{-5}$) is two orders of magnitude smaller than that of the first-order Semi-implicit Euler ($\sim 10^{-3}$). The trajectory and phase-space

plots (Figs. 2 and 5) reveal that Semi-implicit Euler precesses; this is a phase error arising because the numerical solution corresponds exactly to a perturbed Hamiltonian with a slightly different orbital period. Leapfrog, being second-order, reduces this phase error enough that no precession is visible over 100 orbits.

6.3 Computational Efficiency Vs. Accuracy

Computational Efficiency is an important consideration, Since when simulating large clusters of N-bodies and or long running simulations time efficiency is critical. Computational Efficiency is dependent on the number of force evaluations per step.

Table 4: Comparison of integrator properties: order, symplecticity, cost, and long-term error behaviour over 100 orbital periods.

Integrator	Order	Force evals.	Energy error	Ang. mom. error
Explicit Euler	1	1	unbounded (secular)	$\sim 5 \times 10^{-5}$
Semi-implicit Euler	1	1	$\sim 10^{-3}$	$\sim 10^{-14}$
Leapfrog	2	1	$\sim 10^{-5}$	$\sim 10^{-15}$
RK4	4	4	secular ($< 10^{-7}$)	$\sim 2 \times 10^{-10}$

If you want to run a simulation for 10^5 orbits, you need to run rk4 with a much smaller time step to delay secular drift, making it significantly more expensive than leapfrog. Leapfrog achieves geometric stability with a larger step, making it far more efficient in practice.

Leapfrog gives the best compromise: symplectic, cheap, and accurate enough that its energy error is bounded at a very low level.

6.4 Limitations and Sources of Error

Several Simplifications in our study affect the interpretation of the results but do not undermine the main conclusions.

The orbital trajectories in Section 5.1 were produced with a relatively large time step ($\Delta t = T/100$) to make the qualitative differences between integrators immediately visible. The convergence study (Section 4.4) confirms that all methods achieve their nominal orders when the step is sufficiently small;

the qualitative behaviour (spiral, precession, closed orbit) is robust, although the precise error magnitudes would be smaller with a finer step.

We simulated 100 orbital periods, which is sufficient to reveal secular trends in RK4’s energy and angular momentum, but extrapolations beyond this interval (e.g. eventual orbit decay) remain suggestive rather than definitive.

We used a perfectly circular orbit as the initial condition for simplicity. The stability and error behaviour might differ quantitatively for highly eccentric orbits, but the fundamental difference between symplectic and non-symplectic methods remains general.

Only the (x, v_x) projection of the 4-dimensional phase space is shown in Fig. 5. This projection captures the essential dynamical structure (closed vs. open curves) and is sufficient to illustrate the integrators’ geometric properties, but a complete portrait would require all four dimensions.

7 Conclusion

We compared four numerical integrators for the two-body problem — Explicit Euler, Semi-implicit Euler, Leapfrog, and RK4 — and evaluated their ability to preserve the geometric structure of the orbit over 100 orbital periods. The results separate the methods into two clear classes.

Explicit Euler, a non-symplectic first-order method, is conditionally unstable for oscillatory dynamics and causes the orbit to spiral outward. RK4, despite its high fourth-order accuracy, exhibits a slow secular drift in both energy and angular momentum; over 100 orbits the drift is small enough that the trajectory appears closed, but it would eventually destroy the orbit. In contrast, the symplectic Semi-implicit Euler bounds the energy error and prevents secular growth in angular momentum, although its first-order nature leads to a noticeable energy oscillation ($\sim 10^{-3}$) and visible orbital precession. Leapfrog combines second-order accuracy with symplecticity: the energy error is bounded at $\sim 10^{-5}$, angular momentum is conserved to machine precision, and no precession is discernible. Leapfrog achieves this with a single force evaluation per time step, making it both geometrically faithful and computationally efficient.

The central finding of this study is that ****symplecticity is more important than formal order of accuracy**** for long-term geometric fidelity in gravitational simulations. A low-order symplectic integrator preserves the

orbit's shape and conserved quantities indefinitely, while even a high-order non-symplectic method will eventually fail. Leapfrog, or its higher-order symplectic extensions, should therefore be the default choice for any simulation where the conservation of energy and angular momentum over many dynamical timescales is essential.

8 Appendix

8.1 Integrator implementations

The simulation advances the state vector $\mathbf{y}_i = (x_i, y_i, v_{x,i}, v_{y,i})^T$ for each body using the following four integrators. All methods share the same acceleration routine (see Listing 8.1.1) and differ only in how they update positions and velocities.

8.1.1 Acceleration calculation

```

1 def compute_accelerations(self):
2     for b in self.bodies:
3         b.ax, b.ay = 0.0, 0.0
4
5     for i in range(len(self.bodies)):
6         for j in range(i + 1, len(self.bodies)):
7             b1, b2 = self.bodies[i], self.bodies[j]
8             dx, dy = b2.x - b1.x, b2.y - b1.y
9             dist_sq = dx**2 + dy**2
10            if dist_sq == 0: continue
11
12            dist = math.sqrt(dist_sq)
13            force = self.G * b1.mass * b2.mass / dist_sq
14
15            fx = force * (dx / dist)
16            fy = force * (dy / dist)
17
18            b1.ax += fx / b1.mass
19            b1.ay += fy / b1.mass
20            b2.ax -= fx / b2.mass
21            b2.ay -= fy / b2.mass

```

Listing 1: Compute accelerations from Newton’s law of gravity.

8.1.2 Explicit Euler (non-symplectic)

```
1 def _step_euler_NONSYMPLECTIC(self):
2     self.compute_accelerations()
3     for b in self.bodies:
4         b.x += b.vx * self.dt
5         b.y += b.vy * self.dt
6         b.vx += b.ax * self.dt
7         b.vy += b.ay * self.dt
```

Listing 2: Explicit Euler step.

8.1.3 Semi-implicit Euler (symplectic)

```
1 def _step_euler_SYMPLECTIC(self):
2     self.compute_accelerations()
3     for b in self.bodies:
4         b.vx += b.ax * self.dt
5         b.vy += b.ay * self.dt
6         b.x += b.vx * self.dt
7         b.y += b.vy * self.dt
```

Listing 3: Semi-implicit Euler step – velocity updated first.

8.1.4 Leapfrog (Velocity Verlet)

```
1 def _step_leapfrog(self):
2     # Half kick
3     for b in self.bodies:
4         b.vx += 0.5 * b.ax * self.dt
5         b.vy += 0.5 * b.ay * self.dt
6     # Drift
7     for b in self.bodies:
8         b.x += b.vx * self.dt
9         b.y += b.vy * self.dt
```

```

10     # Update accelerations at new positions
11     self.compute_accelerations()
12     # Second half kick
13     for b in self.bodies:
14         b.vx += 0.5 * b.ax * self.dt
15         b.vy += 0.5 * b.ay * self.dt

```

Listing 4: Leapfrog / Velocity-Verlet step.

8.1.5 Runge–Kutta 4 (non-symplectic)

```

1 def _step_rk4(self):
2     # Save original state
3     for b in self.bodies:
4         b.orig_x, b.orig_y = b.x, b.y
5         b.orig_vx, b.orig_vy = b.vx, b.vy
6
7     # k1
8     self.compute_accelerations()
9     for b in self.bodies:
10         b.k1vx = b.ax * self.dt
11         b.k1vy = b.ay * self.dt
12         b.k1x = b.vx * self.dt
13         b.k1y = b.vy * self.dt
14
15     # k2
16     for b in self.bodies:
17         b.x = b.orig_x + 0.5 * b.k1x
18         b.y = b.orig_y + 0.5 * b.k1y
19         b.vx = b.orig_vx + 0.5 * b.k1vx
20         b.vy = b.orig_vy + 0.5 * b.k1vy
21     self.compute_accelerations()
22     for b in self.bodies:
23         b.k2vx = b.ax * self.dt
24         b.k2vy = b.ay * self.dt
25         b.k2x = (b.orig_vx + 0.5 * b.k1vx) * self.dt
26         b.k2y = (b.orig_vy + 0.5 * b.k1vy) * self.dt
27
28     # k3
29     for b in self.bodies:

```

```

30         b.x = b.orig_x + 0.5 * b.k2x
31         b.y = b.orig_y + 0.5 * b.k2y
32         b.vx = b.orig_vx + 0.5 * b.k2vx
33         b.vy = b.orig_vy + 0.5 * b.k2vy
34     self.compute_accelerations()
35     for b in self.bodies:
36         b.k3vx = b.ax * self.dt
37         b.k3vy = b.ay * self.dt
38         b.k3x = (b.orig_vx + 0.5 * b.k2vx) * self.dt
39         b.k3y = (b.orig_vy + 0.5 * b.k2vy) * self.dt
40
41     # k4
42     for b in self.bodies:
43         b.x = b.orig_x + b.k3x
44         b.y = b.orig_y + b.k3y
45         b.vx = b.orig_vx + b.k3vx
46         b.vy = b.orig_vy + b.k3vy
47     self.compute_accelerations()
48     for b in self.bodies:
49         b.k4vx = b.ax * self.dt
50         b.k4vy = b.ay * self.dt
51         b.k4x = (b.orig_vx + b.k3vx) * self.dt
52         b.k4y = (b.orig_vy + b.k3vy) * self.dt
53
54     # Final weighted average
55     for b in self.bodies:
56         b.x = b.orig_x + (b.k1x + 2*b.k2x + 2*b.k3x + b.
57             k4x) / 6.0
58         b.y = b.orig_y + (b.k1y + 2*b.k2y + 2*b.k3y + b.
59             k4y) / 6.0
60         b.vx = b.orig_vx + (b.k1vx + 2*b.k2vx + 2*b.k3vx
61             + b.k4vx) / 6.0
62         b.vy = b.orig_vy + (b.k1vy + 2*b.k2vy + 2*b.k3vy
63             + b.k4vy) / 6.0

```

Listing 5: Fourth-order Runge–Kutta step (synchronous).

References

- [1] Burden, R. L., & Faires, J. D. (2011). *Numerical analysis* (9th ed.). Brooks/Cole.
- [2] Goldstein, H., Poole, C., & Safko, J. (2002). *Classical mechanics* (3rd ed.). Addison-Wesley.
- [3] Hairer, E., Lubich, C., & Wanner, G. (2006). *Geometric numerical integration* (2nd ed.). Springer.
- [4] Wisdom, J., & Holman, M. (1991). Symplectic maps for the n -body problem. *The Astronomical Journal*, **102**, 1528–1538. <https://doi.org/10.1086/115978><https://doi.org/10.1086/115978>
- [5] Yoshida, H. (1990). Construction of higher order symplectic integrators. *Physics Letters A*, **150**(5–7), 262–268. [https://doi.org/10.1016/0375-9601\(90\)90092-3](https://doi.org/10.1016/0375-9601(90)90092-3)[https://doi.org/10.1016/0375-9601\(90\)90092-3](https://doi.org/10.1016/0375-9601(90)90092-3)